Programowanie Autonomicznych Robotów Mobilnych

Robot OmniDrive z kołami omnikierunkowymi

Konspekt teoretyczny · wersja 1.1, 2026.10.31

---

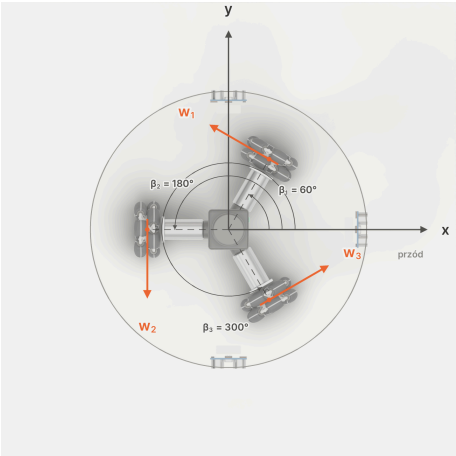
1 Wstęp

Aby robot mógł dojechać do zadanego punktu, musi znać własne położenie. Żaden z czujników robota nie mierzy go jednak bezpośrednio i bez błędu. Robot OmniDrive porusza się na trzech **kołach omnikierunkowych**, a do określania położenia może wykorzystać enkodery kół (**odometrię**), skaner laserowy **LiDAR** oraz trzy pary czujników odległości: **podczerwieni (IR)** i **ultradźwiękowe (US)**, skierowane do przodu, w lewo i w prawo. Konspekt omawia model ruchu robota, źródła informacji o jego położeniu i ich błędy, a następnie **filtr Kalmana**, który łączy te źródła w jedną, dokładniejszą estymatę.

2 Model ruchu robota omnikierunkowego

2.1 Koła omnikierunkowe i napęd holonomiczny

Na obwodzie koła omnikierunkowego znajdują się bierne rolki. Koło napędza robota wyłącznie w kierunku stycznym do okręgu, na którym leżą koła, a w kierunku prostopadłym swobodnie przetacza się na rolkach. Trzy koła rozstawione co 120° pozwalają więc jechać w dowolnym kierunku bez skręcania i jednocześnie, niezależnie, obracać się wokół własnej osi. Taki napęd nazywamy **holonomicznym**: robot ma trzy stopnie swobody na płaszczyźnie (x, y, θ) i może sterować każdym z nich niezależnie. Samochód lub robot z napędem różnicowym nie pojedzie natomiast wprost w bok.



Rozmieszczenie kół i kierunki, w których każde z nich napędza robota

2.2 Dwa rodzaje ruchu

Ruch robota opisuje prędkość w jego własnym układzie: (v_x, v_y) (oś x do przodu, oś y w lewo) oraz prędkość obrotu ω. Rozróżniamy dwa niezależne rodzaje ruchu:

- **Ruch postępowy** (translacja), $\omega = 0$: robot przesuwa się z prędkością v w kierunku φ, czyli $v_x = v\cos\varphi$, $v_y = v\sin\varphi$, nie zmieniając orientacji.
- **Ruch obrotowy** (rotacja), $v = 0$: robot obraca się w miejscu z prędkością ω; wszystkie koła kręcą się wtedy z tą samą prędkością.

Koło ustawione pod kątem β_i (60°, 180°, 300°) w odległości L od środka robota porusza się obwodem z prędkością

$$w_i = \underbrace{-\sin\beta_i\, v_x + \cos\beta_i\, v_y}_{\text{ruch postępowy}} + \underbrace{L\,\omega}_{\text{ruch obrotowy}} .$$

Prędkość każdego koła jest sumą udziałów obu ruchów, dlatego można je wykonywać jednocześnie, np. jechać po prostej i równocześnie się obracać (**ruch złożony**). Trzy takie równania tworzą układ $\mathbf{w} = \mathbf{M}\mathbf{v}$ (**kinematyka odwrotna**), z którego sterownik wyznacza prędkości kół. Prędkość koła jest ograniczona, więc gdy suma udziałów przekracza maksimum, wszystkie koła są proporcjonalnie spowalniane i robot porusza się wolniej, niż zadano.

2.3 Układ odniesienia

W ruchu złożonym istotne jest, względem czego liczony jest kierunek jazdy φ. W układzie **robota** (robot-centric) kierunek „do przodu” obraca się razem z robotem, więc obracający się robot zatacza łuk. W układzie **świata** (field-centric) sterownik na bieżąco obraca zadany wektor o aktualny kąt robota θ:

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} v_x^{\text{świat}} \\ v_y^{\text{świat}} \end{bmatrix},$$

dzięki czemu robot jedzie po prostej, niezależnie od tego, jak się obraca. Kąt θ pochodzi z odometrii, więc jego błąd i opóźnienie zakrzywiają tor jazdy.



3 Odometria

Odometria działa odwrotnie do kinematyki odwrotnej: z dróg $\Delta s_1, \Delta s_2, \Delta s_3$ przebytych przez obwody kół wyznacza przesunięcie robota w jego układzie, $[\Delta x, \Delta y, \Delta\theta]^T = \mathbf{M}^{-1}[\Delta s_1, \Delta s_2, \Delta s_3]^T$, i dodaje je do położenia, obracając o kąt robota w połowie kroku, $\bar{\theta} = \theta + \Delta\theta/2$:

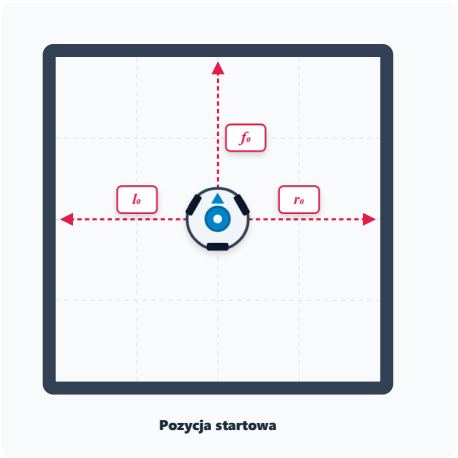
$$x \leftarrow x + \Delta x\cos\bar{\theta} - \Delta y\sin\bar{\theta}, \qquad y \leftarrow y + \Delta x\sin\bar{\theta} + \Delta y\cos\bar{\theta}, \qquad \theta \leftarrow \theta + \Delta\theta.$$

Odometria jest płynna i dostępna w każdym kroku, ale nic jej nie koryguje: każdy błąd (poślizg koła, niedokładny promień, nierówne podłoże) sumuje się i położenie **dryfuje**. Szczególnie szkodliwy jest poślizg podczas obrotu, bo błąd kąta θ obraca każde kolejne przesunięcie.

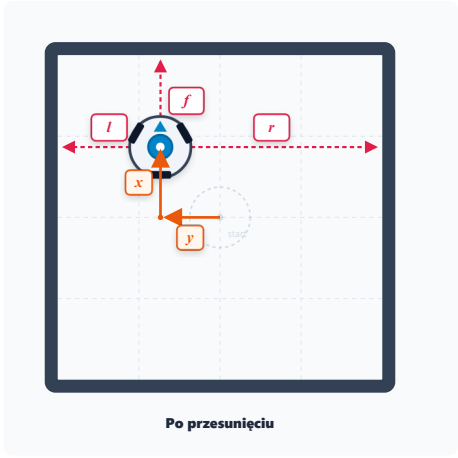
4 Położenie na podstawie LiDAR-u

LiDAR mierzy około 680 odległości w polu widzenia 240°. Uśrednienie kilku sąsiednich punktów w osi przedniej, lewej i prawej daje trzy odległości od ścian: f, l, r. W klatce 78 × 78 cm, przy stałej orientacji robota, położenie względem startu (indeks 0) wynosi

$$x = f_0 - f, \qquad y = \frac{(l_0 - l) - (r_0 - r)}{2}.$$



Położenie startowe



Po przesunięciu o x i y

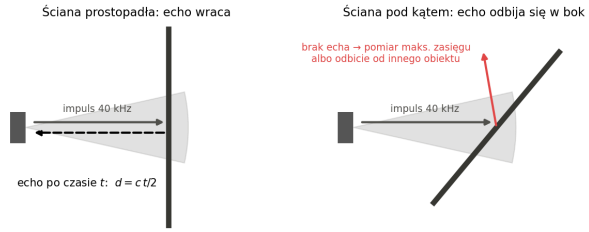
Takie położenie nie dryfuje, bo ściany się nie przesuwają, ale jest zaszumione i działa tylko w znanym otoczeniu. W dowolnym otoczeniu roboty budują z kolejnych skanów mapę i dopasowują do niej każdy nowy skan (*scan matching*).

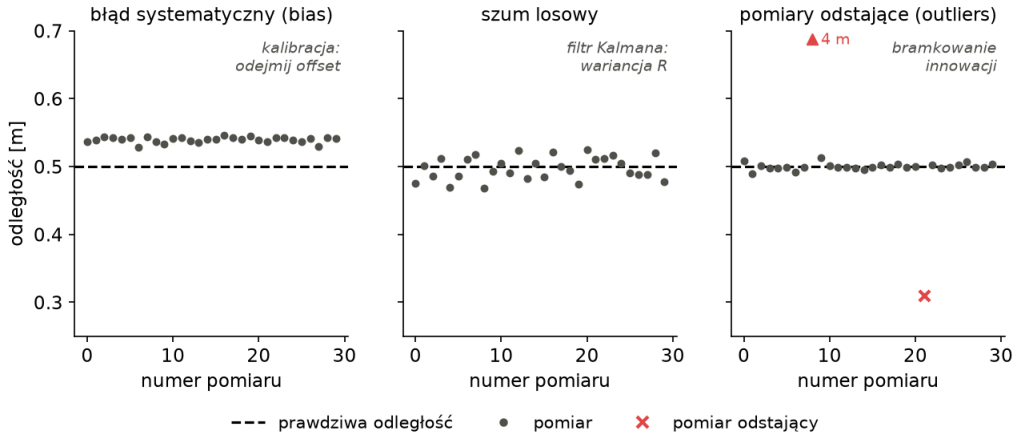
5 Czujniki odległości i błędy pomiaru

Czujnik **podczerwieni (IR)** VL53L1X wysyła impuls światła i mierzy czas jego powrotu; mierzy w wąskim stożku z dokładnością kilku milimetrów. Czujnik **ultradźwiękowy (US)** HC-SR04 wysyła impuls 40 kHz i mierzy czas t powrotu echa. Dźwięk pokonuje drogę do przeszkody dwukrotnie, więc

$$d = \frac{c\,t}{2}, \qquad c \approx 343\ \mathrm{m/s}.$$

Fala odbita od ściany ustawionej pod kątem może nie wrócić do czujnika; wtedy czujnik nie zwraca odczytu albo zwraca echo od innego obiektu.





Błędy pomiaru dzielą się na trzy rodzaje: **błąd systematyczny** (bias) usuwa się kalibracją, **szum losowy** uśrednia filtr, a **pomiary odstające** (outliers) należy wykryć i odrzucić albo ograniczyć ich wpływ.

6 Szum procesu Q i szum pomiaru R

Niepewność obu rodzajów źródeł opisujemy wariancjami. **Szum procesu** Q to wariancja błędu odometrii w jednym kroku. Błąd pojedynczego kroku jest zbyt mały, by go zmierzyć, ale wariancje niezależnych błędów sumują się, więc po N krokach dryf Δ (np. różnica odometrii i LiDAR-u na mecie) spełnia

$$\sigma^2_{\text{całk}} \approx N\,Q \quad \Longrightarrow \quad Q \approx \frac{\Delta^2}{N}.$$

Szum pomiaru R to wariancja błędu pojedynczego odczytu czujnika. Wyznaczamy go z błędu względem odniesienia (LiDAR-u), $e = z_{\text{czujnik}} - z_{\text{lidar}}$. Zwykła wariancja jest bardzo wrażliwa na pojedyncze fałszywe echa, dlatego stosujemy odporną estymację przez **MAD** (medianę odchyleń bezwzględnych):

$$\text{MAD} = \text{median}\big(|e - \text{median}(e)|\big), \qquad \sigma \approx 1{,}4826 \cdot \text{MAD}, \qquad R = \sigma^2.$$

7 Filtr Kalmana

7.1 Filtr Kalmana w jednym wymiarze

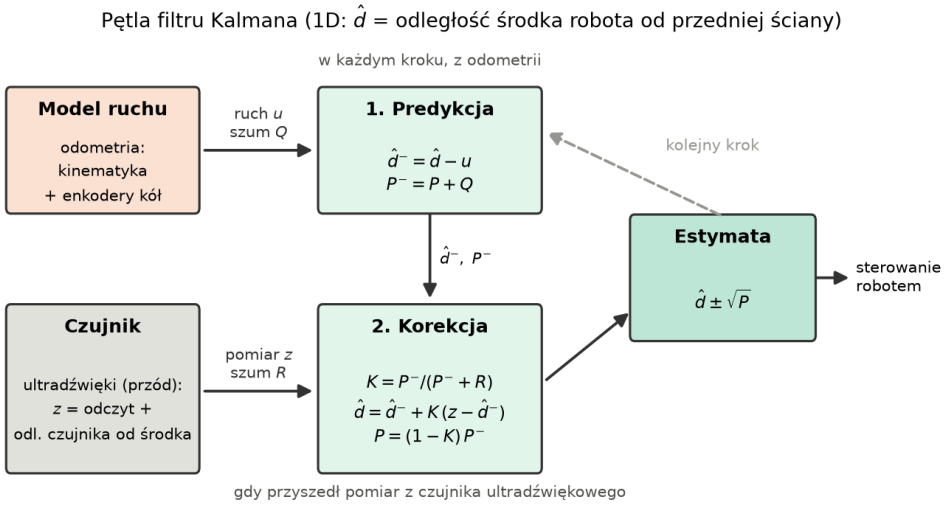
Odometria jest płynna, ale dryfuje; czujniki odległości nie dryfują, ale są zaszumione. **Filtr Kalmana** łączy oba źródła. Przechowuje estymatę stanu x (położenia robota) i jej wariancję P, wykonując w każdym kroku t naprzemiennie dwa kroki, począwszy od stanu z poprzedniego kroku x_{t-1}, P_{t-1}.

Predykcja: stan przesuwa się o przemieszczenie z odometrii Δx_t, a niepewność rośnie o Q. Wynik $\bar{x}_t$ to stan przed uwzględnieniem pomiaru:

$$\bar{x}_t = x_{t-1} + \Delta x_t, \qquad \bar{P}_t = P_{t-1} + Q.$$

Korekcja: pomiar musi opisywać tę samą wielkość co stan, więc odległość od ściany d_t zamieniamy na położenie względem startu, $z_t = d_0 - d_t$. Wzmocnienie $K_t \in [0, 1]$ określa stopień zaufania do pomiaru:

$$K_t = \frac{\bar{P}_t}{\bar{P}_t + R}, \qquad x_t = \bar{x}_t + K_t\,(z_t - \bar{x}_t), \qquad P_t = (1 - K_t)\,\bar{P}_t.$$



- Gdy czujnik mocno szumi ($R \gg \bar{P}_t$), $K_t \to 0$: pomiar jest ignorowany, filtr polega na odometrii.
- Gdy czujnik jest precyzyjny ($R \ll \bar{P}_t$), $K_t \to 1$: filtr przyjmuje pomiar.

- Każda korekcja zmniejsza niepewność: $P_t < \bar{P}_t$.

Kolejne czujniki można dołączyć, wykonując po predykcji kilka korekcji po kolei, każdą z własnym pomiarem z i szumem R (**fuzja sekwencyjna**).

7.2 Filtr Kalmana w dwóch wymiarach

W ogólnym przypadku stan i pomiary są wektorami, a wariancje macierzami kowariancji. Dla stanu $\mathbf{x} = [x, y]^T$ i m czujników filtr wykorzystuje:

- **macierz szumu procesu** $\mathbf{Q}_{2\times 2} = \mathrm{diag}(Q, Q)$,
- **macierz szumu pomiaru** $\mathbf{R}_{m\times m} = \mathrm{diag}(R_1, \ldots, R_m)$,
- **macierz obserwacji** $\mathbf{H}_{m\times 2}$, która opisuje, jak każdy czujnik widzi stan: $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{v}$. Każdy wiersz odpowiada jednemu czujnikowi, np. przedni mierzy x, czyli $[1, 0]$, lewy mierzy y, czyli $[0, 1]$, a prawy $-y$, czyli $[0, -1]$.

Wzory mają tę samą postać co w 1D, z macierzami w miejsce liczb:

$$\bar{\mathbf{x}}_t = \mathbf{x}_{t-1} + \Delta\mathbf{x}_t, \qquad \bar{\mathbf{P}}_t = \mathbf{P}_{t-1} + \mathbf{Q},$$

$$\mathbf{K}_t = \bar{\mathbf{P}}_t \mathbf{H}^T \left(\mathbf{H}\bar{\mathbf{P}}_t\mathbf{H}^T + \mathbf{R}\right)^{-1}, \qquad \mathbf{x}_t = \bar{\mathbf{x}}_t + \mathbf{K}_t\left(\mathbf{z}_t - \mathbf{H}\bar{\mathbf{x}}_t\right), \qquad \mathbf{P}_t = \left(\mathbf{I} - \mathbf{K}_t\mathbf{H}\right)\bar{\mathbf{P}}_t.$$

Jakość filtru zależy od proporcji Q i R. Zbyt duże R (lub zbyt małe Q) sprawia, że filtr ufa odometrii i przejmuje jej dryf; zbyt małe R (lub zbyt duże Q) – że podąża za każdym pomiarem i przenosi jego szum.

Literatura

- K. Lynch, F. Park, *Modern Robotics*, rozdz. 13 (kinematyka robotów kołowych).
- G. Welch, G. Bishop, *An Introduction to the Kalman Filter*, UNC TR 95-041.
- S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, rozdz. 3 i 7.
- R. Labbe, *Kalman and Bayesian Filters in Python*, github.com/rlabbe.